

owis_find_deal_engine

Architecture, processes and request flow — explained step by step

Item	Value
What	Product search service: one product name + one country → matching offers from the online shops that deliver to that country
Status	Phase 1 (search service) complete · CI green · ready for local test
Code	github.com/owisdev/claude-ai-clould-repo, folder Owis_Find_Deal_Engine/ , branch claude/cloud-vs-local-7xi988
Language / runtime	Go 1.26 · Docker · Redis · SearXNG
Date	4 October 2026

How to read this document

- Sections 1-2 give the big picture: what the service does and which parts it has.
- Sections 3-8 follow one search request through the system, step by step.
- Sections 9-11 cover running it: the API, configuration and Docker.
- Section 12 lists what comes next; section 13 is a short glossary of the technical words.

1. What the service does

A user types a product name (for example “samsung s pen”) and picks their country. The service searches the online shops that **deliver to that country** and returns one merged list of offers: title, link, short description, image and, when available, the price.

Key ideas

- **Country-aware:** only shops that deliver to the user's country are searched (eBay is skipped for Jordan, Amazon Saudi Arabia is used for KSA).
- **Login required:** every search comes from a signed-in user (Google, Apple or Facebook through an auth provider such as Firebase).
- **As cheap as possible:** a free self-hosted search engine (SearXNG) is used first; paid sources only when needed: SerpApi with Google Shopping (prices) as backup, and optional Apify scrapers for Temu and SHEIN (real prices).
- **Cache:** answers are remembered for up to 24 hours and refreshed automatically, so popular searches cost nothing and prices stay reasonably current.
- **Fair use:** each user has a daily number of searches by plan (free / pro). Answers from the cache are free and never reduce that number.
- **Robust:** if one shop or search source fails, the user still gets the rest; timeouts, fallbacks and safe defaults everywhere.

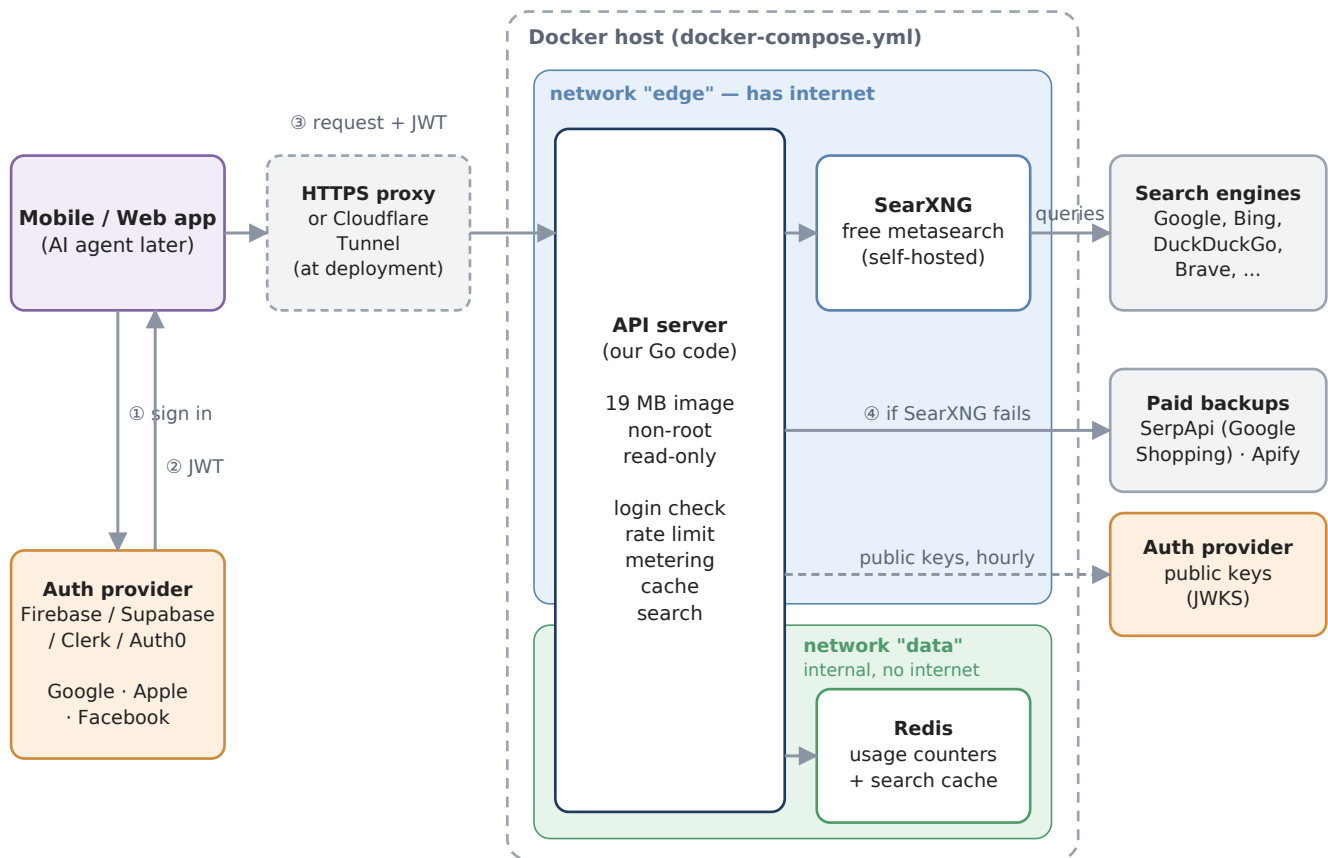
Supported countries and shops

Shop	USA (usa)	Saudi Arabia (ksa)	Jordan (jor)
Amazon	amazon.com	amazon.sa	amazon.com
eBay	ebay.com	ebay.com	— (no delivery)
AliExpress	aliexpress.com	aliexpress.com	aliexpress.com
Temu	temu.com	temu.com	temu.com
SHEIN	us.shein.com	ar.shein.com	ar.shein.com

This table lives in one configuration file (`server/internal/markets/markets.json`). Adding a country or a shop means editing that file — no code change.

2. The big picture

Three programs run together in Docker: the **API server** (our Go code), **SearXNG** (free search) and **Redis** (fast storage for counters and the cache). Around them are outside services: the auth provider that logs users in, the public search engines, and SerpApi as a paid backup.



Only the API port is published. SearXNG and Redis cannot be reached from outside Docker.

Figure 1 — System overview. Numbers show the order of a first search.

The parts

Part	Role	Why it is there
Mobile / Web app	Shows the search box and results	Built later (phase 2+)
Auth provider	Signs users in with Google / Apple / Facebook and issues a signed token (JWT)	We never store passwords; free tier
API server	Our Go service: checks the user, applies limits, searches, merges and caches results	The product itself
SearXNG	Self-hosted metasearch: asks Google, Bing, DuckDuckGo, Brave... in one go	Free search results
SerpApi	Paid Google API; uses Google Shopping (prices, seller, image)	Backup when SearXNG is blocked
Apify (optional)	Store of ready-made scrapers; one per shop for Temu and SHEIN	Real prices where shops have no official API
Redis	In-memory database: daily search counters + cached answers	Fast, shared by all server copies

Part	Role	Why it is there
HTTPS proxy / Tunnel	Adds HTTPS in front of the API in production	Only public entry point (deployment step)

3. Inside the API server

A search request passes through a stack of small layers. Each layer does one job and hands the request to the next one. This keeps every piece simple to test and to change.

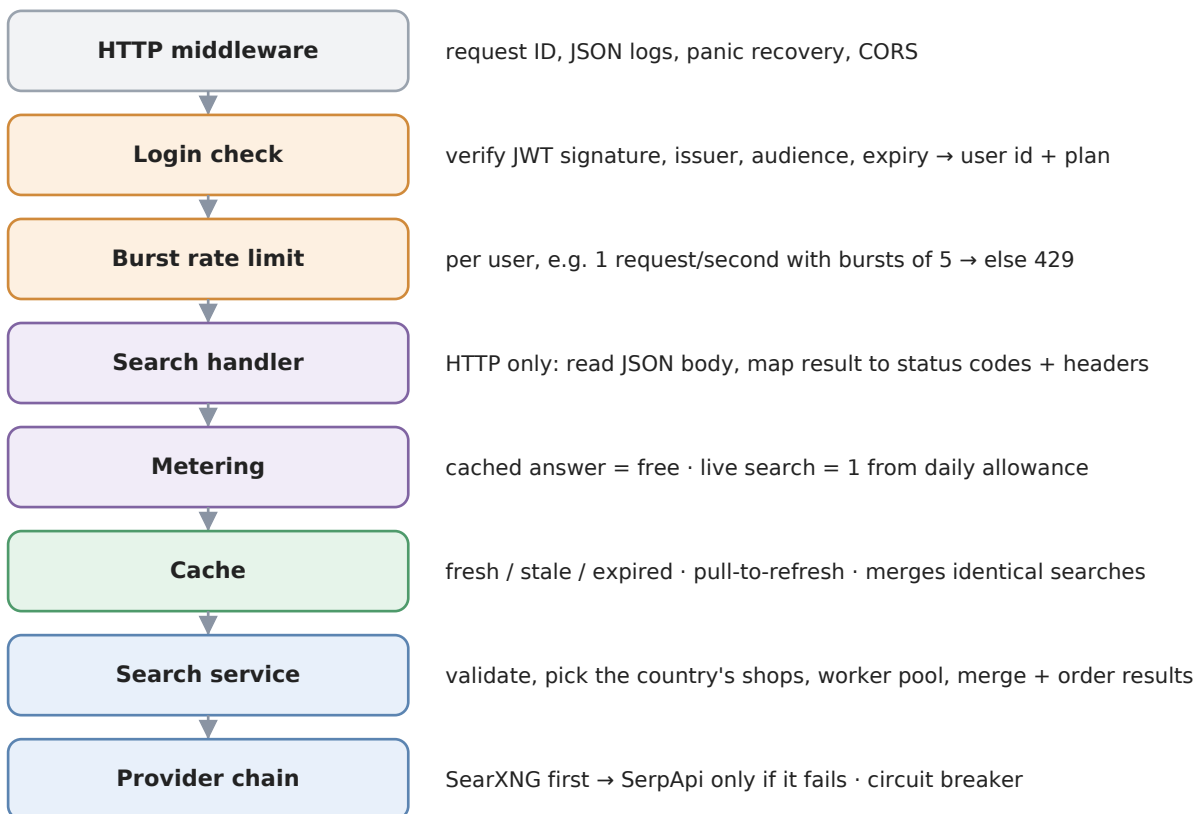


Figure 2 — Layers a search request passes through, top to bottom.

Code map (folder server/)

Folder	What it contains
cmd/owis_find_deal_engine	Program start: reads settings, connects everything, graceful shutdown, health check command
internal/api	HTTP routes, handlers, middleware (login, rate limit, logs, CORS)
internal/auth	JWT verification and cache of the provider's public keys
internal/metering	The rule “cached = free, live = counted”
internal/usage	Plans (free / pro) and daily counters (Redis or memory)
internal/cache	Search cache with freshness rules (Redis or memory)
internal/search	Search service, worker pool, provider fallback chain
internal/providers	SearXNG and SerpApi clients
internal/markets	Countries → shops table (markets.json)
internal/ratelimit	Per-user burst limiter
internal/config	All settings from environment variables, with validation
deploy/searxng	SearXNG settings (Google/Bing enabled, JSON output on)

4. A search request, step by step

This is the complete decision path of `POST /api/v1/search`. Every possible answer the app can receive is on this page.

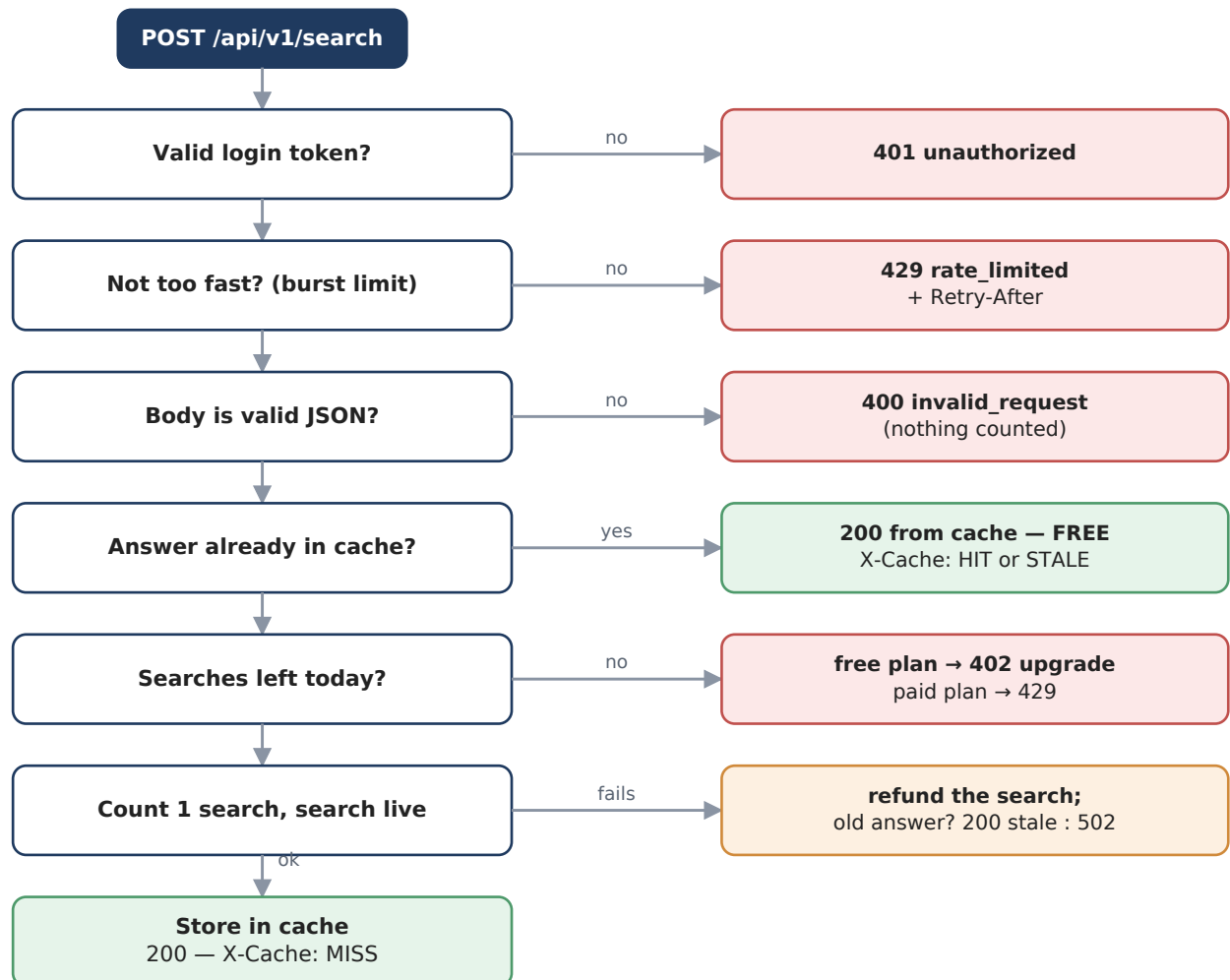


Figure 3 — Decision flow of one search. Left: the checks in order. Right: what the app receives when a check stops the request.

The same steps in words

- Login check.** The token in the Authorization header must be valid and not expired. Otherwise: **401**.
- Speed check.** A user cannot fire requests faster than the burst limit (default 1 per second, bursts of 5). Otherwise: **429** with a wait time.
- Input check.** The body must be JSON with a title (2–200 characters) and a known country code. Otherwise: **400**. Nothing is counted.
- Cache check.** If the answer is cached, it is returned immediately and is **free** — even if the user has no searches left today.
- Allowance check.** Only for live searches: one search is taken from today's allowance. Free plan used up: **402** (show an upgrade screen). Paid plan used up: **429** until midnight UTC.
- Live search.** The shops of the country are searched (section 6). The answer is stored in the cache and returned: **200**, X-Cache: MISS.
- If the live search fails,** the search is given back to the user. If an older cached answer exists it is returned (marked stale); otherwise **502**.

5. Login and security

The server never sees passwords. Users sign in with an **auth provider**; the provider gives the app a **JWT** — a small signed text that says “this is user X, valid until time T”. The app sends it with every request.

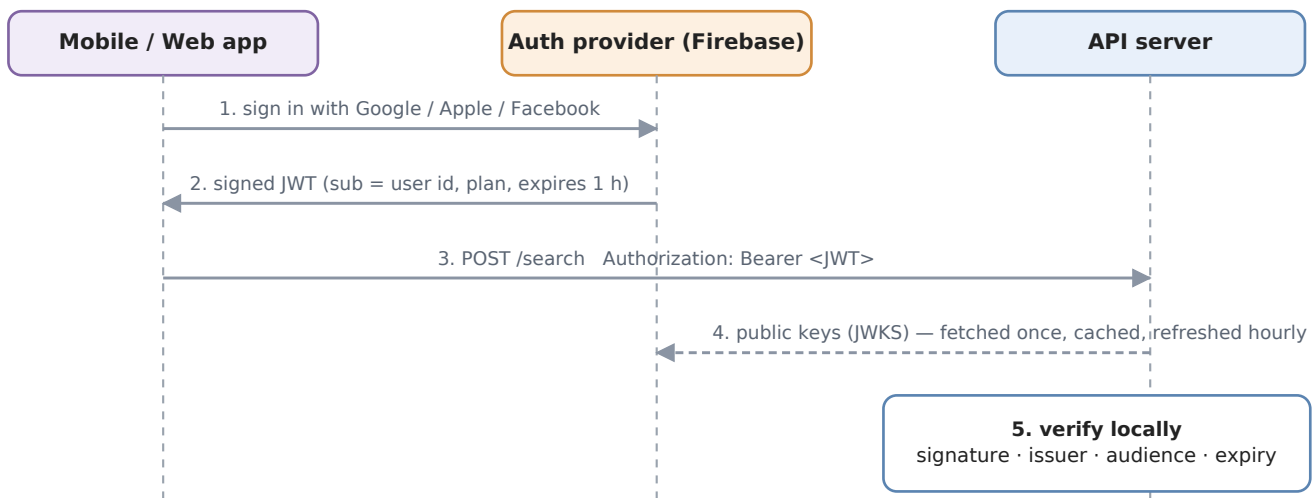


Figure 4 — Login sequence. Step 4 happens once at start and then hourly, not on every request.

What the server checks on every request

- **Signature** — made with the provider's private key; checked with its public keys (JWKS). A changed token fails. Only strong key types are accepted (RS256 / ES256); unsigned tokens and shared-secret tokens are rejected.
- **Issuer and audience** — the token must come from our provider project.
- **Expiry** — expired tokens are refused (Firebase tokens live 1 hour; the app refreshes them automatically).
- **User id and plan** — taken from the token (sub and a plan claim; missing plan = free).

Other protections

- Per-user burst limit and daily allowance (section 8).
- Strict input: unknown JSON fields, oversized bodies and bad titles are rejected.
- Secrets (SerpApi key, Redis password) only in environment variables, never in code or logs; the SerpApi key is removed even from error messages.
- If the counter store (Redis) is down, live searches are refused (**503**) rather than allowed without limits — protects the paid search budget.
- Docker: every container runs as a normal (non-root) user, with a read-only file system, no special Linux permissions and memory limits. Redis has no internet and a password.

6. How the search itself works

Step 1 — pick the shops

The country code selects the shops from the table in section 1. Jordan → Amazon, AliExpress, Temu, SHEIN.

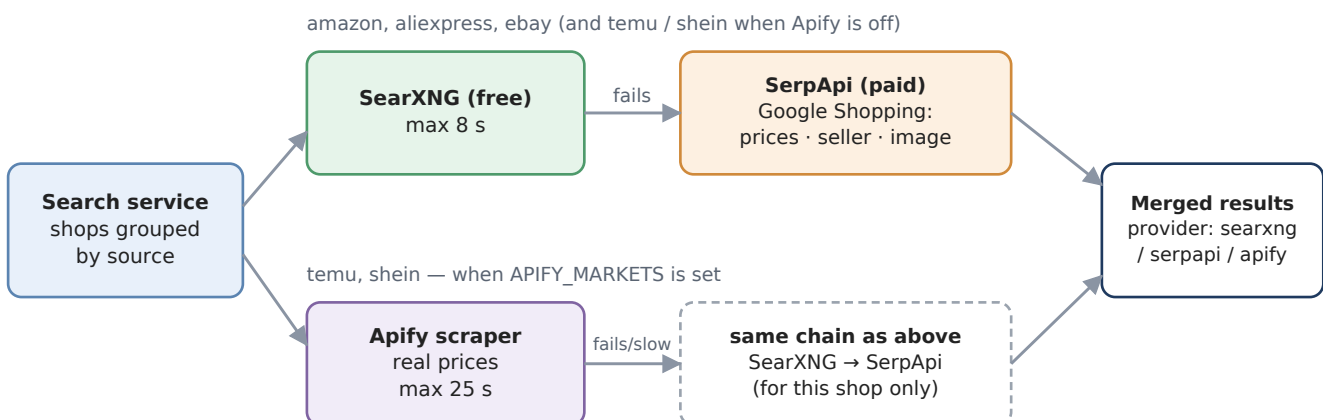
Step 2 — build one query

All shops go into a single web search using the `site:` operator, so one search covers every shop (cheapest option):

```
samsung s pen (site:amazon.com OR site:aliexpress.com OR site:temu.com OR site:ar.shein.com)
```

(Setting `SEARCH_COMBINED=false` sends one query per shop instead, in parallel: better coverage, more cost.) Google Shopping does not support `site:`; there the query is just the product name and offers are matched to our shops by seller name ("Amazon.com", "SHEIN"...).

Step 3 — ask the providers, free first



The first source that succeeds answers. Circuit breaker: 3 failures in a row → that source is skipped for 1 minute.

Figure 5 — Search sources. Paid sources run only when the free one errors, times out or is blocked; Apify (optional) gives Temu and SHEIN real prices.

Step 4 — clean and merge the results

- Each result link is matched to its shop by domain (regional sites such as `sa.shein.com` count as SHEIN); results from other sites are dropped.
- Duplicates are removed, and results are numbered per shop (1st Amazon, 1st Temu, ...).
- The list is ordered so the **best result of every shop comes first**, then the second of each, and so on.
- Missing fields never crash the service; a price is included when the search engine provides one.

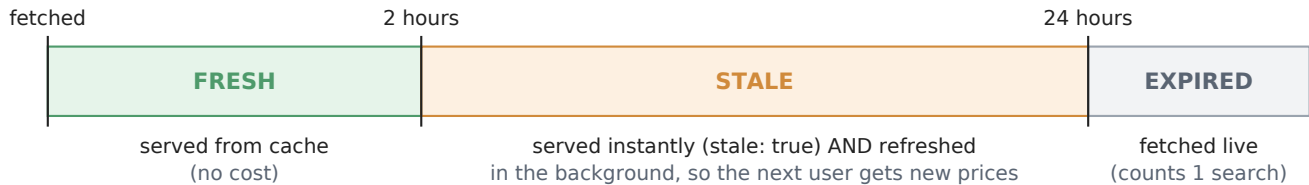
Step 5 — report per shop

The answer includes `markets: ok` or `error` for each shop. One failing shop does not fail the search; the app can show "SHEIN temporarily unavailable". Only if every shop fails does the user get 502.

Concurrency, in short: provider calls run on a small pool of goroutines (Go's lightweight threads) with one overall time limit (15 s; raised automatically when Apify is on). Channels are sized so no goroutine can get stuck, and a crash inside a provider is caught and reported as an error instead of stopping the server.

7. Cache and keeping prices current

Every answer is stored under **country + product name** (upper/lower case and extra spaces do not matter, so “Samsung S PEN” and “samsung s pen” share one entry). What happens next depends on the answer's age:



Age of the cached answer →

Figure 6 — Life of a cached answer (default times; all configurable).

Rules that keep prices honest

Rule	What it does
Background refresh	Stale answers are served instantly and refreshed behind the scenes. At most 4 refreshes run at once.
Shorter life for doubtful answers	If a shop failed, the answer is fresh for 10 min only; an empty answer for 30 min.
Pull to refresh	"refresh": true forces a live search, at most once per 10 min per product (prevents abuse).
Old answer beats an error	If a live search fails but an older answer exists, the older one is returned with stale: true.
One search for many users	If 20 users search the same product at the same moment, only one live search runs.
Shop list changes	Editing markets.json automatically stops old answers from being used.
Shown to the user	Every answer has fetched_at, cached and stale, so the app can show “prices updated 3 h ago”. The shop link always opens the live price.

Later (phase 2): a scheduled job re-checks items saved in users' carts and sends price-drop notifications; eBay and AliExpress results (exact prices from their APIs) get their own shorter cache time.

8. Plans and daily allowance

Each user has a plan; each plan has a number of **live** searches per day (UTC). Defaults: **free** = 20, **pro** = 500 (PLANS=free:20,pro:500).

Situation	Counted?	Answer to the app
Answer found in cache	No — free	200, X-Cache: HIT / STALE
Live search, allowance left	Yes, 1	200, X-Cache: MISS
Live search fails	No — given back	502 (or older answer, stale)
Invalid input	No	400
Free plan used up, not cached	—	402 payment_required → show upgrade
Paid plan used up, not cached	—	429 quota_exceeded + Retry-After
Counter store down, not cached	—	503 (refused to protect the budget)

Every search answer carries these headers so the app can show “N searches left today” without spending one:

Header	Meaning
X-Plan	plan name, e.g. free
X-RateLimit-Limit	searches per day for this plan
X-RateLimit-Remaining	live searches left today
X-RateLimit-Reset	when the counter resets (Unix time, midnight UTC)
X-Cache	MISS (live) · HIT (fresh cache) · STALE (cache, being refreshed)
X-Request-ID	id of this request, also in the server logs (for support)

Where the plan comes from: a plan field in the login token (for example a Firebase custom claim set after payment). Unknown or missing = free. The counting rule lives in one place (the metering layer), so a future AI-agent endpoint will follow the same rule automatically.

9. API reference

Method	Path	Login	Purpose
GET	/api/v1/health	no	Is the server up?
GET	/api/v1/countries	no	Countries and their shops
POST	/api/v1/search	yes	Search a product in a country

Search request

POST /api/v1/search

Authorization: Bearer <JWT from the auth provider>

Content-Type: application/json

```
{ "title": "samsung s pen", "country": "jor", "refresh": false }
```

Search answer (200)

```
{
  "query": "samsung s pen",
  "country": "jor",
  "results": [
    { "market": "amazon", "title": "Samsung Galaxy S Pen ...",
      "link": "https://www.amazon.com/...", "snippet": "...",
      "thumbnail": "https://...", "price": 29.99, "currency": "$",
      "position": 1, "provider": "searxng" },
    { "market": "temu", "title": "...", "position": 1, "provider": "searxng" }
  ],
  "markets": { "amazon": "ok", "aliexpress": "ok", "temu": "ok", "shein": "error" },
  "took_ms": 812,
  "fetched_at": "2026-10-02T12:00:00Z",
  "cached": false,
  "stale": false
}
```

Errors

Always the same shape: {"error": {"code": "...", "message": "..."}}

Status	code	Meaning / what the app should do
400	invalid_request, unsupported_country	Fix the input
401	unauthorized	Sign in again / refresh the token
402	payment_required	Free searches used up → upgrade screen
429	rate_limited	Too fast → wait Retry-After seconds
429	quota_exceeded	Paid plan's daily searches used up
502	upstream_error	All shops failed → “try again”
503	service_unavailable	Temporary problem → retry later
500	internal_error	Bug → report with X-Request-ID

10. Configuration

All settings are environment variables, read at start and checked: the server refuses to start with a clear message if something required is missing or wrong. The full list with comments is in `server/.env.example`.

Setting	Default	Purpose
AUTH_JWKS_URL / AUTH_ISSUER / AUTH_AUDIENCE	— (required)	Which auth provider project to trust
AUTH_PLAN_CLAIM	plan	Token field holding the user's plan
PLANS / DEFAULT_PLAN	free:20,pro:500 / free	Daily live searches per plan
SEARCH_PROVIDERS	searxng,serpapi	Order of search sources
SEARXNG_URL	—	Address of SearXNG
SERPAPI_KEY	—	Needed only if serpapi is listed
SERPAPI_ENGINE	google_shopping	Prices from Google Shopping, or google (web)
APIFY_MARKETS / APIFY_TOKEN	empty	Shops that use an Apify scraper (temu,shein)
APIFY_<SHOP>_ACTOR / _INPUT	—	Which scraper, and its input with {{query}}
APIFY_TIMEOUT / APIFY_MAX_ITEMS	25s / 10	Scraper time limit / results per run
SEARCH_COMBINED	true	One query for all shops (cheapest)
SEARCH_TIMEOUT	15s	Max time for a whole search
PROVIDER_ATTEMPT_TIMEOUT	8s	Time SearXNG gets before falling back
PROVIDER_FAILURE_THRESHOLD / COOLDOWN	3 / 1m	Circuit breaker
CACHE_FRESH_TTL / CACHE_STALE_TTL	2h / 24h	Cache life (section 7)
CACHE_PARTIAL_TTL / CACHE_EMPTY_TTL	10m / 30m	Shorter life for doubtful answers
CACHE_MIN_REFRESH	10m	Min time between pull-to-refresh
RATE_LIMIT_RPS / RATE_LIMIT_BURST	1 / 5	Per-user speed limit
REDIS_URL / REDIS_PASSWORD	—	Counters and cache (memory if empty)
SEARXNG_SECRET	—	SearXNG's own secret (Docker)
API_BIND / API_PORT	127.0.0.1 / 3002	Where Docker publishes the API
CORS_ALLOWED_ORIGINS	empty	Websites allowed to call the API

11. Running it: Docker, tests and CI

Full stack with one command

```
cd Owis_Find_Deal_Engine/server
cp .env.example .env          # fill in: AUTH_*, SEARXNG_SECRET, REDIS_PASSWORD
docker compose up -d --build
docker compose ps              # 3 containers "healthy"
curl localhost:3002/api/v1/health
```

Step-by-step instructions for your PC (including how to get a free test login token from Firebase, and PowerShell commands) are in `Owis_Find_Deal_Engine/STATUS.md`.

What Docker sets up

Container	Networks	Published?	Protection
server	edge + data	API port only, on 127.0.0.1	non-root, read-only, no capabilities, 256 MB, health check
searxng	edge	no	runs as its own user (image default is root), read-only
redis	data (no internet)	no	password, non-root, read-only, keeps counters on disk, evicts cache before counters

Startup and shutdown

- At start the server checks all settings, connects to Redis and downloads the auth provider's public keys; any problem stops it with a clear message.
- On stop (`docker compose stop`) it stops accepting requests, finishes running ones and background cache refreshes, then exits (max 10 s).
- Logs are one JSON line per event (request, user id, status, duration, request id).

Quality checks

- 97 automated test functions (128 test cases) across all packages, run with Go's race detector (finds unsafe concurrent code).
- GitHub Actions on every push: formatting, dependency check, vet, tests, known-vulnerability scan, Docker build and smoke test. First run: all green.
- Manually verified end to end with real Redis and SearXNG containers: login, cache hit/miss/stale, refresh, free cache answers, 402, network isolation, graceful stop.

12. What comes next

Phase	Steps	Content
1 — search service	1-6 ✓	Everything in this document
2 — users	7-11	User profile (/me); plans + payment provider; saved cart with notes and price-drop tracking; purchase reports (shop, date, price) with a friendly “did you buy it?” reminder; notifications (in-app + push). Adds Postgres and a Redis job queue.
3 — more sources, launch	12-15	eBay and AliExpress APIs (real prices, affiliate income) when keys are approved; deployment with HTTPS; access for AI agents (OpenAPI / MCP tool).
Future feature	noted	Search by photo (Google Lens): the user takes a photo, the service finds the product and its offers in the shops of their country (same cache, limits and answer format). With the mobile app.

Right now: run the stack locally (STATUS.md), then report back with the feedback checklist in STATUS.md — results per country, response times, any errors, and anything to change in the response format before the app is built on it.

13. Glossary

Word	Meaning
API	The set of web addresses the app calls (e.g. /api/v1/search).
JWT	Signed token proving who the user is; issued by the auth provider.
JWKS	The auth provider's public keys, used to check a JWT's signature.
Cache hit / miss	Answer found in / not in the stored answers.
Stale	A cached answer older than the fresh period, still usable while being refreshed.
Provider	A search source: SearXNG (free), SerpApi or Apify (paid).
Scraper (Apify Actor)	A ready-made program that reads a shop's website and returns products.
Fallback	Using the next provider when the first one fails.
Circuit breaker	Temporarily skipping a source that keeps failing, so it does not slow every request.
Rate limit / allowance	Speed limit per second / number of live searches per day.
Goroutine	A lightweight thread in Go; used to call providers in parallel.
Redis	Very fast in-memory database; holds counters and cached answers.
SearXNG	Open-source search engine that combines results from many search engines.
Docker / compose	Packages each program in a container; compose starts them all together.
CI	Automatic checks on GitHub after every push.
Distroless	A minimal container image without a shell — less to attack.